



In-memory compression of Awkward Arrays with Coffea

Peter Fackeldey¹, Iason Krommydas²,
Princeton University¹, Rice University²



Memory is the most precious Resource...

Workers have limited memory, and if the memory usage of a job exceeds that limit, processing will crash.

Often limited memory results in small chunk sizes for processing, which leads to:

- More overhead from scheduling many small jobs
- Less efficient use of compiled and vectorized code, which performs better on larger data
- Significant overhead from CPython

→ It's crucial for stability and performance to reduce memory usage!

Coffea analysis with Awkward Arrays

Many modern HEP analyses are powered by Awkward Arrays, which are designed to handle complex, variable-length data structures efficiently, and coffea, which provides seamless cluster-scaling and physics-intuitive tools for analysis.

For reducing memory usage we have to first understand how Awkward Arrays are stored in memory.

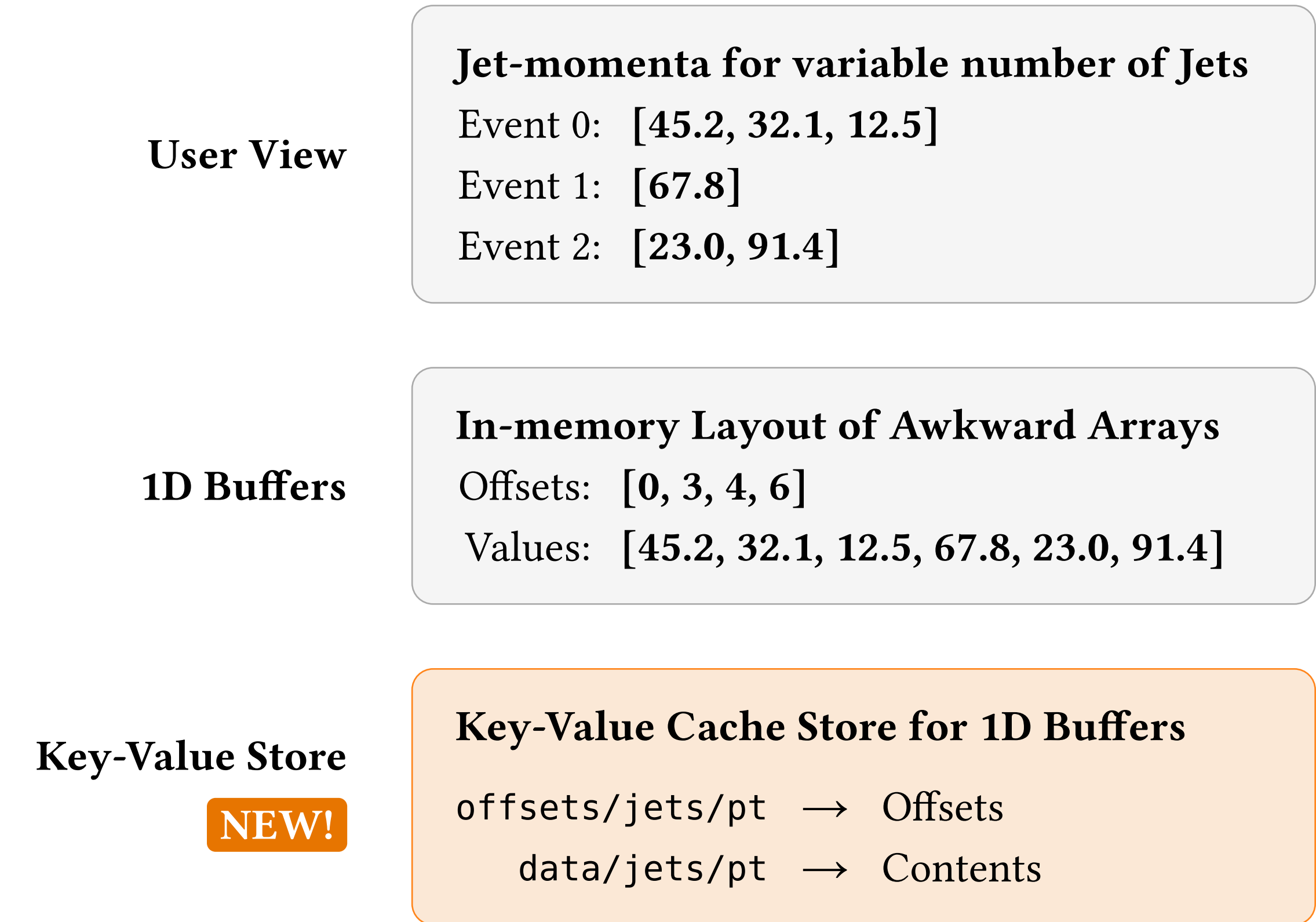
How Awkward Arrays are Stored in Memory

Awkward Arrays represent nested, variable-length data (e.g. per-event lists of particles) as a **collection of flat 1D buffers**. Each variable-length field uses two fundamental buffers:

- **Offsets:** A 1D array of integers defining the start and end positions for each event's data in the values buffer.
- **Values:** A flattened 1D array holding the actual data values for all events, end-to-end.

This means an array of 100k events with a variable number of jets is stored as just two contiguous 1D buffers, rather than a Python list of 100k variable-length arrays.

Crucially, each individual buffer is identified by a **unique buffer key** (a string). This design maps naturally onto a key-value store: the buffer key is the lookup key, and the 1D buffer data is the value, which can be compressed to bytes before storage.



Why this matters: Since each buffer is an independent, contiguous chunk of memory, we can store them individually. *Only buffers that are actually accessed during analysis will be read from the key-value store.* This allows us to implement efficient key-value stores that can reduce the in-memory footprint of Awkward Arrays.

Key-Value Caches for Awkward Arrays

We extended coffea to support a pluggable key-value cache interface for Awkward Array buffers, called BufferCache. This allows users to implement custom caching strategies:

```
1 from coffea.nanoevents import NanoAODSchema, NanoEventsFactory
2 from coffea.nanoevents.mapping import BufferCache
3
4 buffer_cache = BufferCache(...)
5
6 factory = NanoEventsFactory.from_root(
7     {path: "Events"},
8     mode="virtual",
9     buffer_cache=buffer_cache,
10 )
```

In the following, we will discuss different cache implementations, and how to use them with the BufferCache to reduce memory usage in coffea analyses.

In-memory Compression with BufferCache

We can use the BufferCache to implement in-memory compression of Awkward Array buffers. By compressing the buffer data before storing it in the cache, we can significantly

reduce the memory footprint of the arrays. Only upon access are the buffers decompressed and loaded into memory, allowing for larger chunk sizes and more efficient processing without exceeding memory limits.

To use in-memory compression, we can utilize a codec from the numcodecs library, such as Blosc, which provides fast compression and decompression:

```
1 from coffea.nanoevents.mapping import BufferCache
2 from numcodecs import Blosc
3
4 buffer_cache = BufferCache(
5     cache={},
6     codec=Blosc("zstd", clevel=1, shuffle=Blosc.BITSHUFFLE)
7 )
```

On-disk Caching with BufferCache

The most aggressive way to reduce memory usage is to use an on-disk cache. By storing the buffers on disk instead of in memory, we can effectively bypass memory limitations. This allows for processing arbitrarily large datasets, at the cost of increased latency due to disk I/O.

To implement an on-disk cache, we can use a simple file-based approach or leverage libraries like zict for more sophisticated caching strategies.

```
1 from coffea.nanoevents.mapping import BufferCache
2 from numcodecs import Blosc; import zict
3
4 buffer_cache = BufferCache(
5     cache=zict.File("cache_dir"),
6     codec=Blosc("zstd", clevel=1, shuffle=Blosc.BITSHUFFLE)
7 )
```

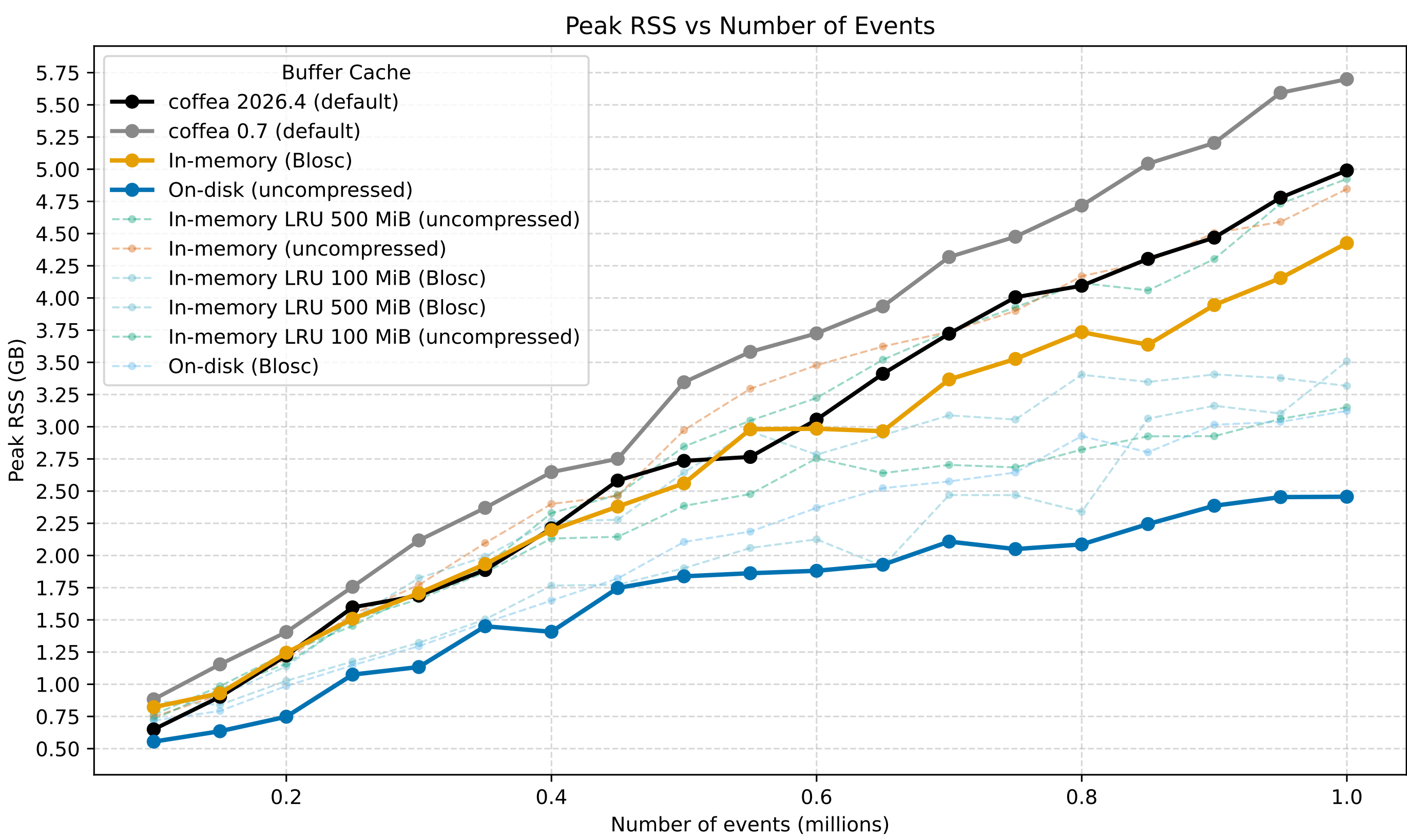
Other Cache Implementations

BufferCache is flexible and allows for any cache implementation that follows the MutableMapping[str, bytes] interface. This means users can implement custom caching strategies, such as tiered LRU caches using zict.LRU.

Benchmarks and Performance

In the following, we will show benchmarks comparing the memory usage and performance of different caching strategies using BufferCache in a typical coffea analysis. We will compare:

- No cache (default) for awkward v1 (coffea 0.7) and v2 (coffea 2027.4)
- In-memory compression with Blosc (In-memory ...)
- On-disk caching with zict.File using SSD with ~10GB/s read speed (On-disk ...)
- Tiered LRU caching with zict.LRU with a threshold in MiB (In-memory LRU ...)



The **peak RSS (resident set size) is significantly reduced** with in-memory compression, and even more so with on-disk caching by up to a factor of ~2× compared to no caching.

It should be noted that these caching strategies come at the cost of slightly increased runtime due to the overhead of compression and disk I/O. In this benchmark, the runtime overhead of these strategies was in the range of ~1..10%. Much of this can be mitigated by increasing the chunk size again (making more use of compiled kernels) and by tuning the cache settings (e.g. compression levels). Especially on-disk caching can lead to significant increases in runtime if the disk bandwidth is low and thus needs to be *carefully assessed per compute infrastructure*.

→ Check out coffea>=2026.4 if you struggle with memory limits in your analysis!